

Topic 12: Hashing and Hash Tables

12.1 What is Hashing?

- A technique to map a key to an array index using a hash function
- Goal: $O(1)$ average time for insert, search, and delete
- Hash function $h(\text{key})$ should distribute keys uniformly across the table
- A simple hash: $\text{index} = \text{key} \% \text{tableSize}$
- Good hash functions minimise collisions (two keys mapping to the same index)

12.2 Collision Resolution - Chaining

- Each array slot holds a linked list of all keys that hash to that index
- Insert: compute $h(\text{key})$, append to the list at that index -- $O(1)$ average
- Search: compute $h(\text{key})$, search the list -- $O(1)$ average, $O(n)$ worst (all keys in one chain)
- Load factor $\lambda = n/m$ (n =keys, m =table size) -- keep $\lambda < 1$ for good performance

```
class HashTable {
    vector<list<int>> table;
    int size;
    int hash(int key) { return key % size; }
public:
    HashTable(int s) : size(s), table(s) {}
    void insert(int key) { table[hash(key)].push_back(key); }
    bool search(int key) {
        for (int k : table[hash(key)])
            if (k == key) return true;
        return false;
    }
};
```

12.3 Collision Resolution - Open Addressing

- All keys stored directly in the array -- no linked lists
- Linear probing: on collision, try next slot ($\text{index}+1$, $\text{index}+2$, ...) -- causes clustering
- Quadratic probing: try $\text{index}+1^2$, $\text{index}+2^2$, ... -- reduces clustering
- Double hashing: use a second hash function for the step size -- best distribution

12.4 STL Unordered Containers

- `unordered_map<K,V>`: hash table -- $O(1)$ average lookup, insert, delete

- `unordered_set<T>`: hash set -- $O(1)$ average contains, insert, delete
 - Load factor managed automatically -- rehashes when $\text{load factor} > 1$
 - Custom hash for user-defined types: provide a `hash<T>` specialisation
-